

Portuguese Song Search System

João Pedro Dias Fernandes
up202108044@fe.up.pt
Faculty Of Engineering of University of Porto
Porto, Porto, Portugal

Igor Monteiro de Andrade
up202108674@fe.up.pt
Faculty Of Engineering of University of Porto
Porto, Porto, Portugal

João Brandão Alves
up202108670@fe.up.pt
Faculty Of Engineering of University of Porto
Porto, Porto, Portugal

Ema Maria Monteiro Martins
up202402794@fe.up.pt
Faculty Of Engineering of University of Porto
Porto, Porto, Portugal

Abstract

With the constant evolution of the internet, the amount of information available on the internet are getting even bigger. With that in mind, to address those big amounts of data, it is important to have specialized search systems to address those areas.

We are focusing on creating a system to search and retrieve information about music in Portuguese that was produced in 2016, as it is one of the biggest entertainment industries in Portugal and, usually, the lyrics are misunderstood by the listeners, where a search system will help in that question.

This article contains all the needed steps to develop a robust search engine to address that area. That contains collecting data from Kaggle and Last.FM, clean, prepare, and characterize data, then we index using schemas on Solr, and then, on some prospective search tasks, we intend to query it and evaluate our results.

CCS Concepts

• **Information systems** → Information Retrieval and Query Processing.

Keywords

Information retrieval, Data preparation, Portuguese, Music, 2016, Natural language processing, Text mining, Search system, Document indexing, Music reviews, Music Stats

ACM Reference Format:

João Pedro Dias Fernandes, João Brandão Alves, Igor Monteiro de Andrade, and Ema Maria Monteiro Martins. 2024. Portuguese Song Search System. In *Portuguese Song Search System*, November 21, 2024, Porto, Portugal. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

In today's world, the huge amount of digital information available around the web has made it necessary to have effective tools for

searching and retrieving relevant data. Our project focuses on creating a search system centred on Portuguese music that was produced and released in 2016 — a year that was rich and diverse in musical output and cultural significance. The choice of language and year came from our feeling of nostalgia for a year that turned out to be full of achievements for our birth country. By collecting and processing data related to this music dataset, we intend to provide a system that can easily search through lyrics, reviews, and other related documents like stories about how the songs were created.

This report details the work done for the first milestone of the project, which is all about preparing the data. Data preparation is a necessary and really important step, as the way we collect and clean up the data will directly influence how well the search system will perform later. In this milestone, we identified two relevant datasets from different sources, verified if the data was reliable, and carried out an initial analysis to better understand their characteristics and group them. By the end of this delivery, we will have a collection of around ten thousand documents that are well organized and ready to be used in the next steps.

This report is structured in the following sections: First, the data extraction, where we describe the initial dataset, the enrichment done, and the final clean + finalization. Secondly, the data preparation, where we describe the data pipeline, which includes data cleaning and restructuring. After that, we characterize the data in Data Characterization. To almost finish, before the conclusion, we prospected some possible search tasks.

2 Data Extraction

Following a search for pertinent datasets, two datasets were found: one in CSV format, obtained from *Kaggle*, and the other in JSON format, gotten from an *API* called *Last.FM*, an *API* that specializes in extracting music statistics to certain users that can link their music player accounts (example: *Spotify*, *Deezer*, *Apple Music*, *Soundcloud*, etc.). That is followed by a brief overview of the datasets that we chose.

2.1 Initial Dataset Characterization

Kaggle's initial dataset had a total of 9050 records of songs, which included attributes such as song title, artist, year of release, tag, number of views, language, and lyrics. This dataset, although no license is exactly attached to it, has a rating of 8.24 in *Kaggle*. The source of this dataset is the *Genius* dataset, where it was applied a model of language recognition of lyrics. The *Last.FM's* dataset

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FEUP, Porto, 21 November 2024,

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

had a way bigger total of records as it holds statistics for way more songs than the ones that we are using (songs in Portuguese from 2016). *Last.FM* grants a limited but enough free license to use it, especially for students. By grouping both datasets, we added some attributes like the number of different listeners, the time play count of the songs, some text content about the song and the artist, an image of the album, and an image of the artist. As some of *Kaggle*'s dataset songs had no match into the *Last.FM*'s dataset, we decided to remove those songs from the final dataset. With this, the final dataset's total number of records turned a little lower than the 9050 that we started with.

2.2 Dataset Enrichment

To upgrade the initial dataset, additional data points were gathered from external sources (*Last.FM*). This included fields such as the number of different listeners, total play counts, tags assigned by listeners, and images related to the song, artist, and album. After this, the dataset grew to include more complex attributes, such as information about the artist and the album. However, there were still some missing values, especially in multimedia fields.

2.3 Data Cleaning and Finalization

Data cleaning was performed to remove incomplete records and reduce noise, resulting in a final cleaned dataset of 7885 entries. This part involved eliminating duplicate records and filling in missing values where possible. The cleaned dataset contained the following key attributes:

- Song metadata: title, artist, year of release, and tags.
- Listener engagement metrics: play counts and different listeners from *Last.FM*.
- Language detection results and textual content like lyrics, artist descriptions, and song stories.
- Multimedia elements: album and artist images.

These final attributes will be used to support some better analyses.

3 Data Preparation

In this section, we are presenting our data pipeline, which explains the processes of both data collection and preparation. This process is **a must** for the project as it ensures that: each item has interesting information from different sources; uniform data; and clean information.

Primarily, we focused on reducing our dataset from *Kaggle* to only **Portuguese songs from 2016**, mainly to ensure the data was within an order of magnitude that would make sense in the context of this project. The selection of language and year originated from our feeling of longing for a year that ended up being filled with accomplishments for our native land. We analysed the dataset and noted that the reduced dataset had some null or "no-content" rows, but we decided to ignore it, as it didn't prejudice the primary focus of this project.

With our dataset reduced, we performed queries to the *Last.FM API*. Those queries were firstly using the **desiredComponent**. **trackInfo** method, but if no match is found, we appealed to the **desiredComponent.search** method and retrieved the best match. However, if, still with the search method, the *API* doesn't find any

results, we simply assign the **null string**. From the previous *API*, we gathered such information as different listeners, play count, the history and content of that song, the resume of the artist, associated tags, the image of the artist and the album, and the album name. Note: **desiredComponent** can be a track, album, or artist.

After the data gathering from the *API*, and adding the new columns to the dataset, we performed some **data preparation**, to achieve a more consistent dataset. Firstly, we noticed that some songs haven't got any matches in the *Last.FM API* (around 12%). As it was not so a big percentage, adding the fact of the database already being in the limit of the expected size (around 9000), we decided to eliminate those songs from our database.

After that, we only have one relevant null entry (not considering *Last.FM* entries), which was the title of a song, simply solved by **manual research and input** (what's the title of the song with these lyrics?). To note that we **eliminated** some columns that are not useful for this project (*language* and *language_ft* - these columns are part of a study of language recognition that was done by the author of the dataset we used as a source; we also eliminated year and id). Also, some columns like tags, content_artist, content_song, and others with information that came from *Last.FM* were renamed to make clear what information is from the *API*. Finally, the double quotes were removed from the features column to make the data simpler.

The result of all that pipeline is the *dataset_finale_cleaned*, a dataset that, although not 100% complete, is clear, simple, and easy to work through.

In Figure 1, we have a diagram of the entire pipeline structure, with the proper phases indicated and with their proper description.

3.1 Conceptual Data Model

In this Figure 22, it is shown the **conceptual data model** of this dataset. Each entity has its elements (those who were extracted from *Last.FM* were identified) and its relations. To note that some songs are considered **singles**, in other words, they are not part of an album, so we have a unique relationship for that case.

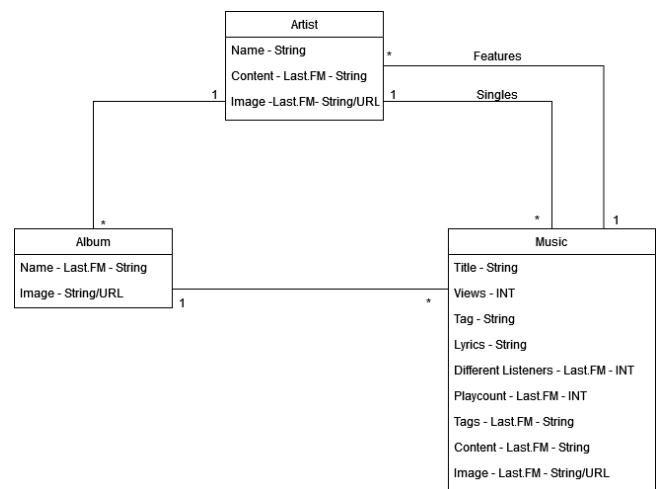


Figure 2: Conceptual Data Model.



Figure 1: Data Pipeline.

Data Clean and Restructure

4.1 Data Types

Name	Type	Example
title	String	Vício
tag	String	rap
artist	String	Blazer Hip-Hop
views	int	12614
features	Collection of Strings	{Tássia Reis}
lyrics	String	Assim como Angola, precisa do PR para governar...
lastfm_different_listeners	int	23797
lastfm_playcount	int	158334
lastfm_tags	Collection of Strings	rap, brazilian, samba, Hip-Hop, seen live
lastfm_content_song	String	Seven is a song recorded by South Korean art...
lastfm_content_artist	String	Os Mind da Gap foram um grupo de hip hop portugueses...
lastfm_image_artist	String	https://www.last.fm/music/Mind+Da+Gap
lastfm_image_album	String	https://www.last.fm/music/Rashid

Table 1: Data Types

The word cloud in Figure 3 represents the main words present in the lyrics of the different songs. Some topics, such as *amor*, *vida*, *casa*, *mundo* are present in the word cloud, which was expected since they are the main topic for a vast part of the portugueses songs.



They are keys for the search tasks needed for the next stage, which is to search for information. In this case, looking for songs based on words present in lyrics is one of the searches that are possible to implement (Figure 5). It can be used, for example, to search for romantic songs. We can use words like "amor", "romance", "beijo," and "abraço" to give more weight to the songs that contain these words. This weight will be used in the process of ranking the retrieved results. Many other searches can be done because they have sufficient content to do so, like finding songs based on the song information (by the *lastfm_content_song attribute*, Figure 4), on the artist/band information (by the *lastfm_content_artist attribute*, Figure 6) and by the song's tags (by the *lastfm_tags* and tag attributes, Figure 7). For example, if the song information speaks about the feeling or the message of the music, it can be used in more abstract searches such as "happy songs" or "relax songs." Another example is that by searching the artist information, if an artist is

Brazilian, the artist has more chance to have funk as a genre of music than a Portuguese one.

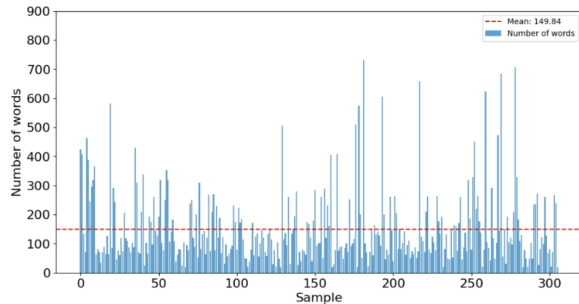


Figure 4: Number of words per song content.

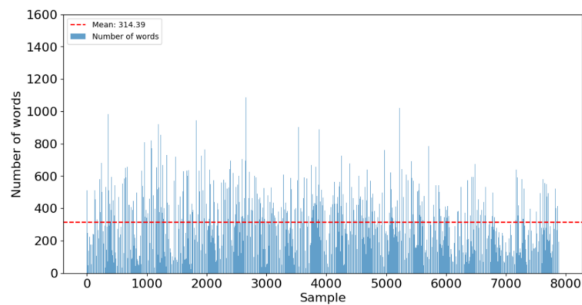


Figure 5: Number of words per lyric.

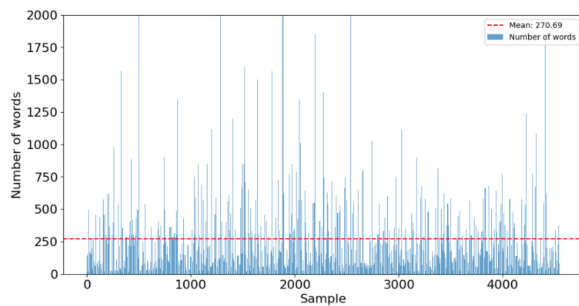


Figure 6: Number of words per artist content.

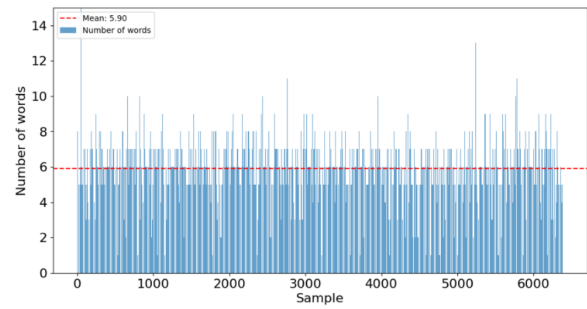


Figure 7: Number of words per *Last.FM* tag.

4.3 Tags distribution

Figure 8 shows the distribution of the tags. The most common tags are *rap*, *pop*, and *rock*. That result is expected once they are the most common styles that people listen to, not only in Portugal but all over the world.

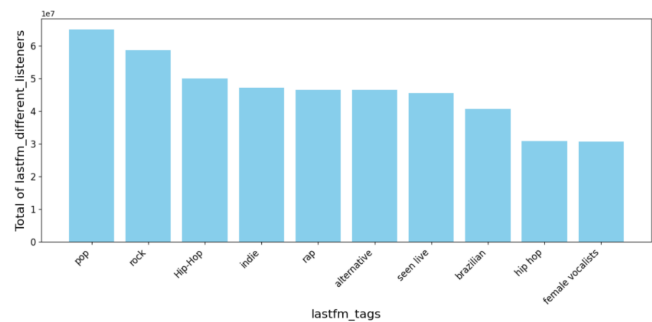


Figure 8: Tags' distribution.

The tags present in Figure 9 show that the most common Last.FM user tags also include *rap*, *pop* and *rock*, which proves the importance of this kind of genre.

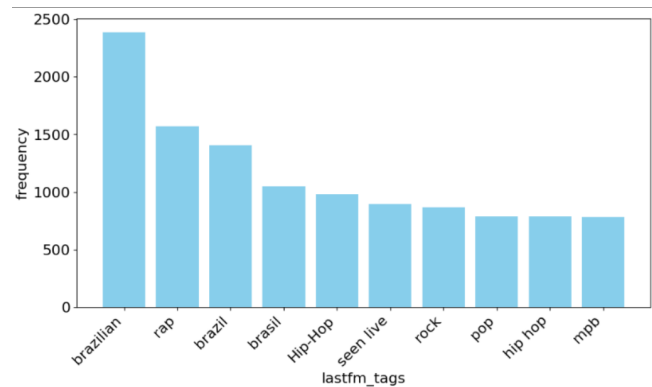


Figure 9: Top ten tags most used by the users.

4.4 Singers/Bands distribution

Figure 10 shows the ten most common singers/bands. In that year, as observed in the table, the singer that had more singles in the dataset was the *MC Igu*.

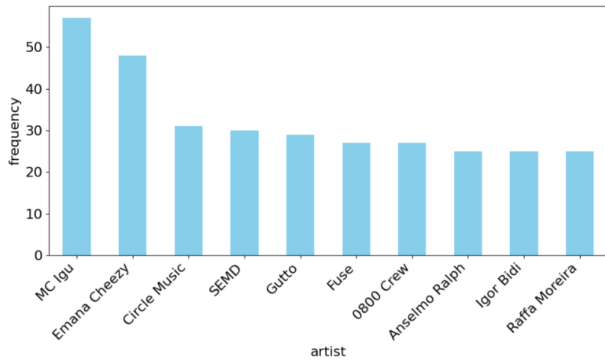


Figure 10: Top ten artists with more songs.

Even though *MC Igu* was the artist with more songs, it wasn't the most listened by *Last.FM* users. The Figure 11 *listeners_artist* shows that *Slim Boy* was the most listened.

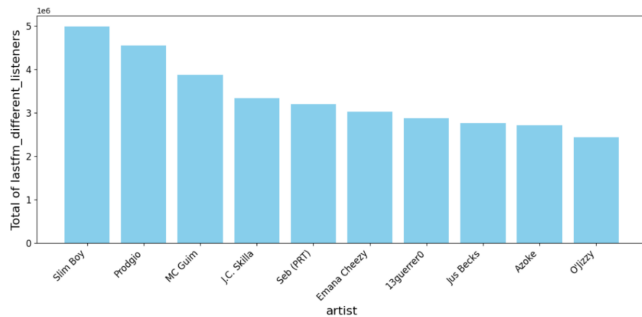


Figure 11: Top ten distribution of artists most listened to by *Last.FM* users.

As represented in Table 2 and Table 3, the tags present in the songs of the artists with more songs and the artists more listened in *Last.FM* match the previous illustrated information, since the most common tags in both cases are rap.

Artist	Tags
MC Igu	rap
Emana Cheezy	rb, misc, rap
Circle Music	pop
SEMD	misc, rap
Gutto	rb
Fuse	rap
0800 Crew	rap
Anselmo Ralph	rb
Igor Bidi	rap
Raffa Moreira	rap

Table 2: Tags related to the songs of the artists with more songs

Artist	Tags
Slim Boy	rb, rap
Prodigio	rb, rap
MC Guim	pop, rap
J.C. Skilla	rap
Seb (PRT)	rap
Emana Cheezy	rb, misc, rap
13guerrero	rap
Jus Becks	rap
Azoke	rap
O'jizy	rap

Table 3: Tags related to the songs of the artists with more listeners in *Last.fm*

5 Prospective Search Task

After this, through analysis of the data collected and the features of said data, it is possible to determine some possible search tasks the user may want to use:

- Find songs with lyrics related to religion.
- Find songs whose artist has hit number 1 in the charts and is Portuguese.
- Find songs whose artist is female that talk about empowerment.
- Find songs that talk about reflection or love.
- Find pop songs that talk about the past.

In a resumed way, we want the users to be able to find any song with any knowledge they have about the song, even if it is only if the song is a *happy* or *sad* song, per example.

6 Data Indexing

Data indexing is a technique that enhances database performance by minimizing the amount of disk I/O (input/output) necessary to retrieve data [?]. This technique is crucial to building an efficient information retrieval system. It is based on documents, which is a result of data restructure.

To help us in this phase, we will use Solr, an open-source search tool written in Java, which offers some reliable tools such as powerful data indexing and querying, supported by a full-text search feature.

In this section, we will present the document definition and the indexing process, which relies on the use of schemas, which we will detail later in this article.

6.1 Document Definition

The documents to be indexed are the result of the data pipeline previously referred, which was collected, prepared, and cleaned for this phase.

Each song comprises a title, an artist, features, views, and a tag, and it is also provided from Last.FM, a play count, number of different listeners, and content of both song and artist. We have some long textual fields, such as the content of the artist and song and also lyrics, which will be useful for these further steps.

6.2 Indexing Process

Indexing a database can be quite troublesome if done ground-up. To help us, Solr provides a set of tokenizers and filters already defined and the possibility to define a schema with those features when uploading data to a core to help us in that task.

Tokenizers are classes that break down text into individual tokens, which are typically words or terms, in order to avoid problems with empty spaces, or so what. Filters take those tokens and apply rules in order to modify, analyze, or enhance. In each field type in a schema, we can apply one tokenizer and various filters. Note that those filters and tokenizers can be applied in the indexing phase (when loading up the data), but also in the querying phase. In our tests, when applying the same tokenizer and filter(s) in both phases, it produced better results.

In our work, we focused on textual fields, preferably long ones, to apply those filters and tokenizers, as it can showcase better our system. As our lyrics field (the focus field of our work) is in Portuguese, it was quite troublesome to find and apply some filters due to language limitations. That said, we used the following tokenizers and filters in our work:

- **StandardTokenizerFactory**: Breaks text into words or expressions to allow a simple rule applying by filters.
- **ASCIIFoldingFilterFactory**: Converts any non-ASCII characters to their ASCII equivalents.
- **LowerCaseFilterFactory**: Converts all uppercase letters to their lowercase version.
- **PortugueseStemFilterFactory**: Reduces Portuguese words to their root form. (Example: casinha -> cas).
- **SynonymGraphFilterFactory**: Expands tokens to include their synonyms. We also defined those synonyms in synonyms.txt

- **FlattenGraphFilterFactory**: Simplifies complex token graphs into a linear sequence for easier processing.

We used all of the above in the user-created-type boostedText and the first three ones in the shortText type, as it wasn't needed to apply such indexing to fields with different language contexts (for example, artists' names). We used the SynonymGraphFilterFactory to allow to accept, in a query, synonyms or terms in the same context of a desired term (example: Jesus -> Deus, ser divino, igreja). It allows us to produce better results in our data context, as music searches by theme can be quite subjective, and maybe, for example, a religious song may not cite Jesus in their lyrics.

The structure of our schema is described below.

Field	Type	Indexed?
lyrics	boostedText	yes
title	boostedText	yes
views	Int	no
artist	shortText	yes
tag	string	yes
features	shortText	yes
lastfm_different_listeners	Int	no
lastfm_playcount	Int	no
lastfm_tags	string	yes
lastfm_content_song	boostedText	yes
lastfm_content_artist	boostedText	yes
lastfm_image_artist	string	no
lastfm_image_album	string	no
album	string	yes

Table 4: Advanced schema fields type definition

7 Retrieval

The retrieval involves using the pre-created information needs along with two different schemas: a simple schema with just default field types and an advanced schema that uses the filters mentioned in the last section. To achieve this, we used some query parameters, where some vary between simple and advanced and others stay the same. The parameters that stay the same independent of the schema are:

- **Query (q)** - Takes the most valuable words of the information need to perform the query;
- **Query Operator (q.op)** - Defines default operator for query expressions (AND or OR);
- **Filter List (fl)** - Limits or adds to the information included in a query response.

These parameters were set as follows:

Parameter	Value
q	religião
q.op	AND
fl	*, score

Table 5: Common query parameters

The value from **q** is the one used for the first information need, used here as an example. Since **q** changes from query to query but not from schema to schema, the value of **q.op** was chosen to be AND as, upon a quick inspection, it presented results better aligned with our vision of the desired outcome when compared to its counterpart OR, and finally the value of **fl** was changed just to give the ability to visualise the score of each one of the results, which was mainly used for finding out the best approach to get the desired results.

For the advanced schemas, we also used its parameters, with the intent to further optimize the search. In the simple search, one of the parameters was also used (**qf**) but without any of the boosts (lyrics title lastfm_content_artist tag artist).

The parameters used were:

- **Query Fields (qf)** - Takes a list of fields, each of which is assigned a boost factor to increase or decrease that particular field's importance in the query;
- **Phrase Fields (pf)** - Used to "boost" the score of documents in cases where all of the terms in the **q** parameter appear in close proximity;
- **Phrase Slop (ps)** - Defines the maximum number of positions one token can be moved in relation to another token in order to match a phrase specified in a query

These parameters were set as follows:

Parameter	Value
qf	lyrics^ 4 title^ 2 lastfm_content_artist^ 3 tag^ 10 artist
pf	lyrics^ 6 lastfm_content_artist^ 6 title^ 10
ps	3

Table 6: Boosted schema parameters

Let's now explain the reasoning for these choices. The **qf** values were chosen based on our vision of the resulting output; if a music genre is mentioned in the query, we want that to be taken into consideration more than any other thing, since we think that if a user searches for an exact music genre, he pretends to find only music of said type, therefore the 10 times boost. This is followed by lyrics and lastfm_content_artist, which are our open text fields and the main focus of our search system, that culminate in having the second highest boosts, with a slight edge to lyrics, which we deemed a bit more important, which are then followed by the title and the artist in this order, which we consider important but not by a lot, resulting in these smaller boost values.

The **pf** were chosen based on a similar criteria, higher priority to lyrics and lastfm_content_artist, as their the main focus of our search, but this time with the addition of a 10 times boost to title in case the word in the query matches in close proximity to title, as we think if a user searches for a title in specific, it wants said song to be presented first. Finally, the **ps** value was chosen to be 3, as we think it is a reasonable value of distance between words to justify applying a boost to the result.

8 Evaluation

One of the most important steps in determining how good an information retrieval system is is to evaluate query performance. In

this case, we examined a series of searches intended to extract pertinent results for a compilation of Portuguese music from a Solr-based search engine. As part of this assessment procedure, queries with various parameters—such as fields and boosting configurations—were run, and the accuracy, recall, precision at 10 (P@10), mean average precision (MAP), and average precision (AvP) metrics obtained from the outcomes were examined. To obtain these metrics, we used trec_eval.

- **Accuracy:** Indicates the percentage of things that were successfully and unsuccessfully retrieved from the total number of items. Because it ignores the imbalance between relevant and non-relevant elements, it is less frequently utilized in information retrieval.
- **Recall:** The percentage of pertinent papers that were recovered from the whole collection of pertinent documents. Fewer pertinent papers are missing when recall is high.
- **Precision:** The percentage of pertinent documents returned. The majority of recovered papers are helpful when accuracy is high.
- **Precision at 10 and 30 (P@10 and P@30):** A targeted precision statistic that assesses how relevant the top 10/30 documents that were retrieved are. For systems that prioritize rankings, this is helpful.
- **Average Precision (AvP):** Highlights the ranking order of pertinent things by calculating the average precision values at each time a relevant document is obtained.
- **Mean Average Precision (MAP):** The mean accuracy of all the evaluation's queries. It offers a single metric that blends ranking and accuracy.

To do the evaluation, we retrieved 30 results per query, a value that we consider reasonable to have a vision about our system. For each of these results, we evaluated manually if the result was relevant or not. With that data, we calculated the precision-recall curves and the MAP. Even though we retrieved 30 results, we consider P@10 instead of P@30; once normally the first 10 results are normally the ones that appear in the first page of a retrieve system.

The subsections below will present the queries and their results. The parameters that are present in the tables are related to the advanced system. To get the simple queries, you only need to take the exponents of numbers in the **qf** parameter and remove the **pf** and **ps** parameters.

8.1 Query 1

Information Need: Songs with lyrics related to religion.

This query was chosen to explore how religious themes, a significant cultural element, are represented in Portuguese music and to assess the system's ability to retrieve songs with this specific focus.

Query:

Parameter	Value
q	religião
q.op	AND
fl	*, score
qf	lyrics^ 4 title^ 2 lastfm_content_artist^ 3 tag^ 10 artist
pf	lyrics^ 6 lastfm_content_artist^ 6 title^ 10
ps	3

Table 7: Parameters for query 1

Evaluation method: In this case, we check if the lyrics are talking about topics like Jesus or some kind of religious references. We notice that if the music has lastfm_tags like gospel, it is a signal that it can be related to religion.

Results: Based on the P@10 results (Table 8), we can conclude that the advanced system has better performance in the first retrieved results. Looking for the P@30 (Table 8), we can see that this advantage increases as we evaluate more files. The AvP (Table 8) demonstrates that the advanced system has better general performance.

Rank	Simple System	Advanced System
P@10	0.5000	0.6000
P@30	0.3333	0.7333
AvP	0.5272	0.7317

Table 8: P@10, P@30, AvP for query 1

As it turns out, two of the precision-recall curves are different. The sharp fall in precision at low recall for the first curve (Figure 12) suggests a poor ability of the model to find positive cases early on. On the other hand, the second curve (Figure 13) shows higher precision at lower recall and, in turn, performs better when capturing positive instances as well. This means that there is a precision-recall trade-off as both curves tend to plateau at higher levels of recall. Nevertheless, the second curve seems to have consistently greater precision overall, implying a higher performance across various recall levels.

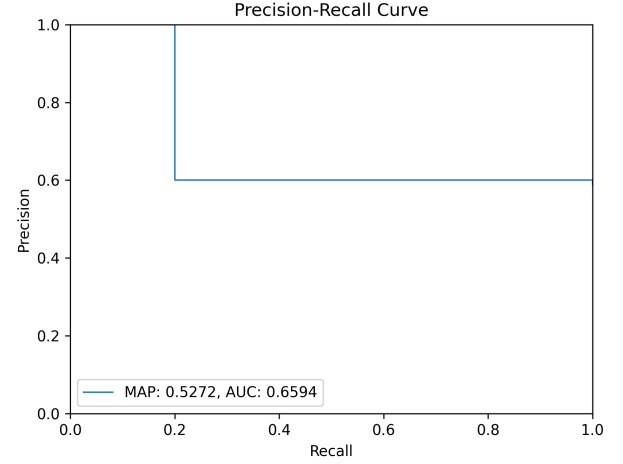


Figure 12: Precision-recall curve for query 1 in simple system

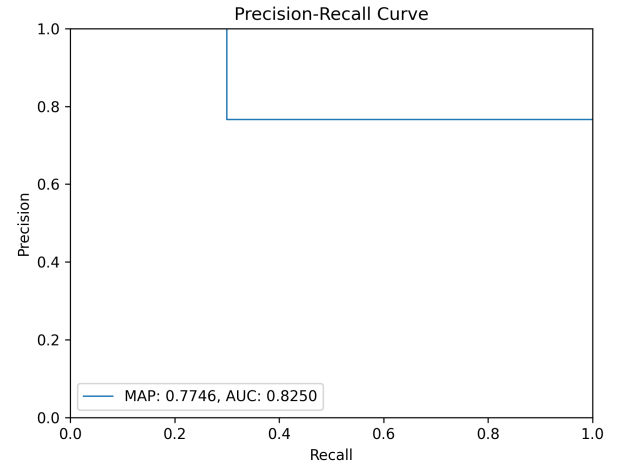


Figure 13: Precision-recall curve for query 1 in advanced system

8.2 Query 2

Information Need: Songs whose artist has hit number 1 in the charts and is Portuguese.

This query was selected to identify top-charting Portuguese artists, showcasing the system's capability to retrieve prominent figures in the national music scene.

Query:

Evaluation method: We determined that to be in number 1, the singer needed to have more than 10 million views. To check if the artist is Portuguese, we saw the lastfm_tags to understand if they have some references to Portugal and also the lastfm_content_artist.

The P@10 (Table 10) demonstrates that the advanced system is completely effective, while the simple system has significantly

Parameter	Value
q	number 1 AND portuguese
q.op	AND
fl	*, score
qf	lyrics^ 4 title^ 2 lastfm_content_artist^ 3 tag^ 10 artist
pf	lyrics^ 6 lastfm_content_artist^ 6 title^ 10
ps	3

Table 9: Parameters for query 2

lower performance. Based on the P@30 (Table 10), the advanced system isn't more completely effective, even though it is much better than the simple system. AvP (Table 10) confirms that the advanced system maintains good results through all the retrieved processes, while the simple system maintains very low and inconsistent performance.

Results:

Rank	Simple System	Advanced System
P@10	0.3000	1.0000
P@30	0.1333	0.6667
AvP	0.3000	1.0000

Table 10: P@10, P@30, AvP for query 2

In this example, you can see a perfect performance as shown by the precision-recall curve (MAP and AUC = 1.0). This means the model has ideal precision (all positive predictions are correct) at all levels of recall. This is a perfect scenario where there are no false positives and negatives (Figure 15). This curve has a lower MAP and AUC than the previous one (Figure 14); therefore, it is suspected that the latter model may be more accurate and robust compared to the former.

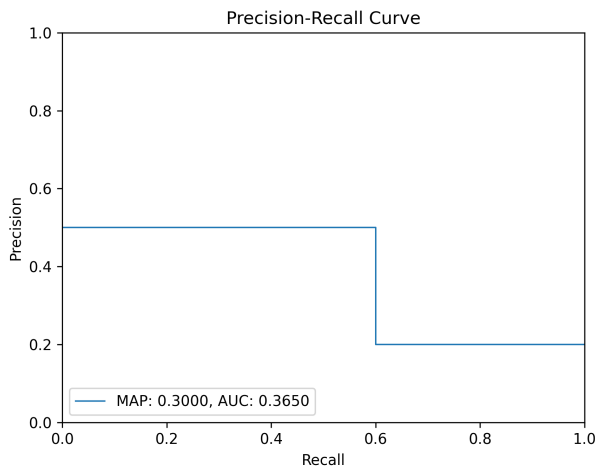


Figure 14: Precision-recall curve for query 2 in simple system

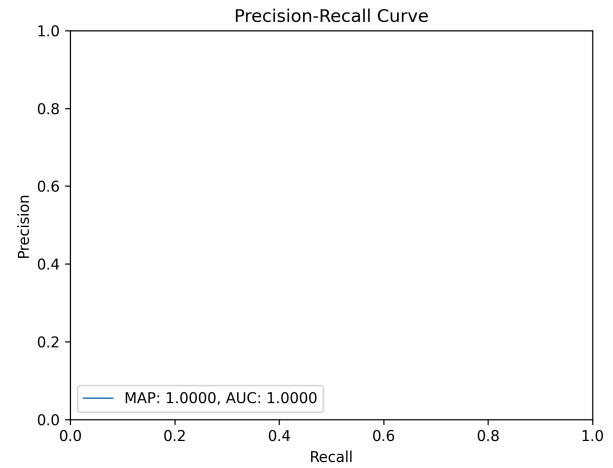


Figure 15: Precision-recall curve for query 2 in advanced system

8.3 Query 3

Information Need: Songs whose artist is female that talk about empowerment.

This query highlights the system's ability to retrieve songs by female artists focused on themes of empowerment, emphasizing diversity and meaningful content.

Query:

Parameter	Value
q	female AND poder
q.op	AND
fl	*, score
qf	lyrics^ 4 title^ 2 lastfm_content_artist^ 3 tag^ 10 artist
pf	lyrics^ 6 lastfm_content_artist^ 6 title^ 10
ps	3

Table 11: Parameters for query 3

Evaluation method: To understand if the artist is female, we look at the lastfm_tags and also for the name of the artists. Related to empowerment, we read the lyrics to see if the content talks about it or has some kind of empowerment speech.

Results: In this query, the results in the simple system were calculated considering only 19 values, due to the number of values that the system retrieved.

By analysing the P@10 (Table 12), the advanced system is much more effective at initial retrieval, indicating a better ability to prioritize relevant documents at the top of the list. With the P@30 (Table 12) we verify that the advanced system maintains high accuracy, while the simple system has very low accuracy, suggesting that the list of documents retrieved by the simple system is full of irrelevant items. Based on the AvP (Table 12), the advanced system shows a very good accuracy, while the simple system seems to be quite inefficient.

Rank	Simple System	Advanced System
P@10	0.1000	0.8000
P@30	0.0667	0.7333
AvP	0.1534	0.8501

Table 12: P@10, P@30, AvP for query 3

The precision-recall curve of the boosted model (Figure 17) has a great enhancement when compared to the simple model (Figure 16). And finally, note that the MAP and AUC values are both significantly higher in the boosted model (i.e., better). The fitted curve starts at high precision levels and gradually decreases with recall increments, which illustrates the precise-perfect balance between precision and recall. This indicates that it identifies positive instances accurately along with covering most of them also. Such a model would be more realistic and practical in a real-world setting than the earlier one.

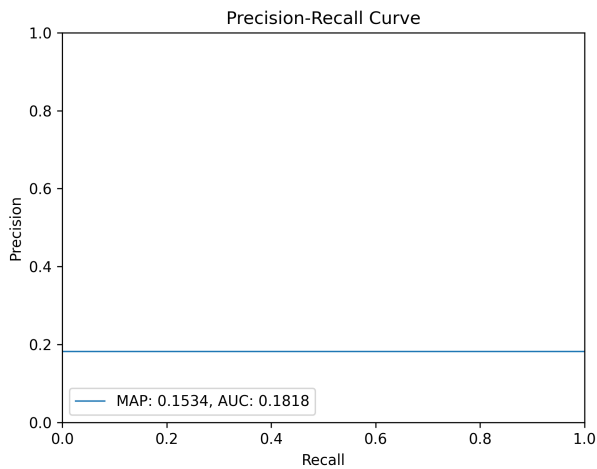


Figure 16: Precision-recall curve for query 3 in simple system

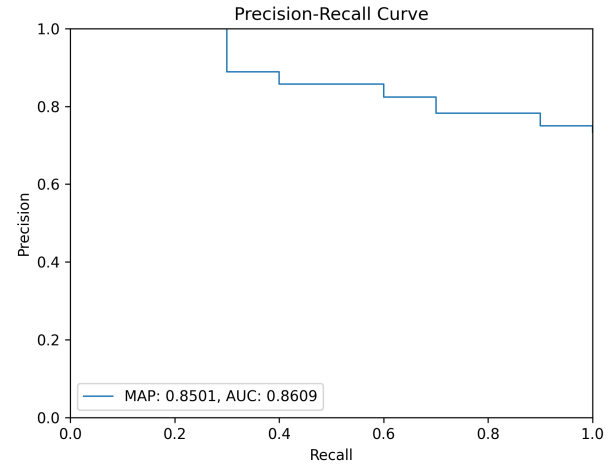


Figure 17: Precision-recall curve for query 3 in advanced system

8.4 Query 4

Information Need: Songs that talk about reflection or love.

This query explores the system's ability to retrieve songs centered on themes of reflection or love, which are universal and deeply relevant topics.

Query:

Parameter	Value
q	reflexão OR amor
q.op	AND
fl	*, score
qf	lyrics^ 4 title^ 2 lastfm_content_artist^ 3 tag^ 10 artist
pf	lyrics^ 6 lastfm_content_artist^ 6 title^ 10
ps	3

Table 13: Parameters for query 4

Evaluation method: In this case, we look for the title, lastfm_tags, the lyrics to see if they talk about reflection or love. Specifically for the reflection, we search for interrogative marks because it is a clue that the lyrics can speak about reflections. We also considered some references to nostalgia, childhood, past relationships, and the future.

Both systems present good values of P@10 and P@30 (Table 14), thus presenting good performance. The advanced system is slightly better, but the difference is not very significant. AvP (Table 14) proves that these two systems are very similar in terms of performance, with the advanced system being a little better.

Results:

Rank	Simple System	Advanced System
P@10	0.6000	0.8000
P@30	0.6333	0.7333
AvP	0.7326	0.8087

Table 14: P@10, P@30, AvP for query 4

Both models have a similar model showing its precision-recall curve—both have good performances. Both MAP and AUC are high, meaning we have a good balance between precision and recall in these cases. Having a high precision level at the start and gradually decreasing when recall increases makes sense (Figure 19), as the model is able to retain positive instances but cannot maintain accuracy (Figure 18). Yet again, there is a little more drop in precision at high recall when comparing the boosted model curve to the simple model curve, suggesting a larger tradeoff between precision and recall here.

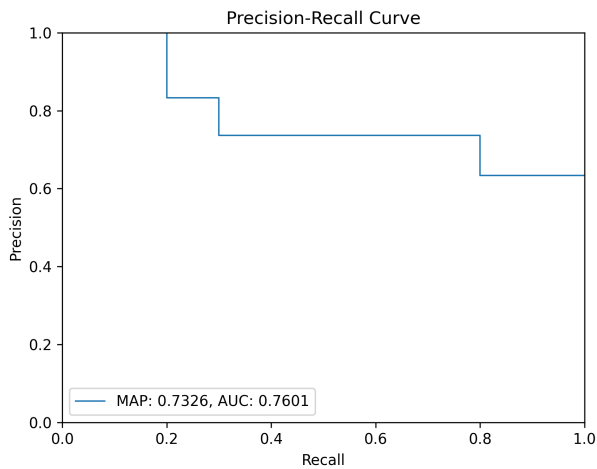


Figure 18: Precision-recall curve for query 4 in simple system

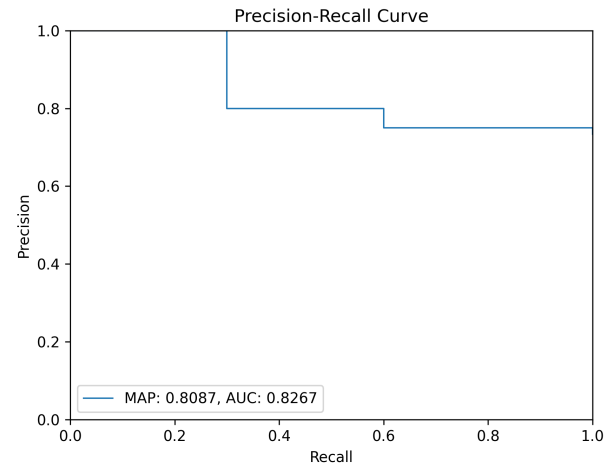


Figure 19: Precision-recall curve for query 4 in advanced system

8.5 Query 5

Information Need: Pop songs that talk about the past.

This query focuses on retrieving pop songs with themes about the past, showcasing the system's capacity to handle genre-specific and thematic searches.

Query:

Parameter	Value
q	pop AND passado
q.op	AND
fl	*, score
qf	lyrics^ 4 title^ 2 lastfm_content_artist^ 3 tag^ 10 artist
pf	lyrics^ 6 lastfm_content_artist^ 6 title^ 10
ps	3

Table 15: Parameters for query 5

Evaluation method: To check the type of the song, we saw the lastfm_tags and the tag. To see if the content is about the past, we read the lyrics, and we tried to find out if the artist is remembering something that is appended in the past or if he misses someone or something.

Both the simple system and the advanced system have low P@10 values (Table 16), showing that the first files are not very relevant. With P@30 (Table 16), values improve, although they remain quite low. AvP (Table 16) shows that the advanced system is much better than the simple system, although, overall, none of them are performing very well.

Results:

Rank	Simple System	Advanced System
P@10	0.1000	0.4000
P@30	0.2000	0.4667
AvP	0.1851	0.4216

Table 16: P@10, P@30, AvP for query 5

Both models have mediocre performance (Figure 21 and Figure 20). The curves show a shallow straight line then decrease in precision value at large recall; this indicates the trade-off between precision and recall. By making changes in class imbalance, training data/high-dimensionality of the dataset, or optimizing model architecture.

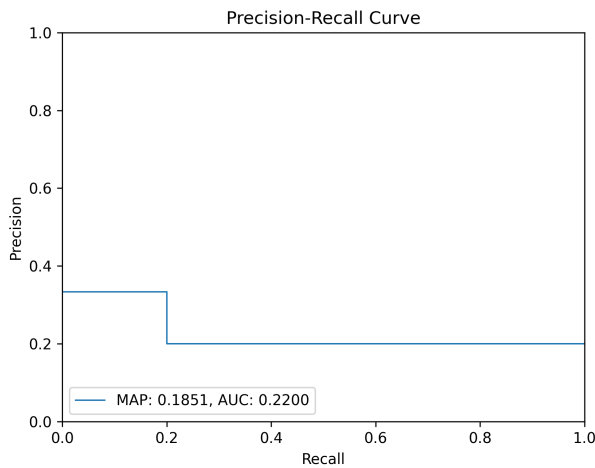


Figure 20: Precision-recall curve for query 5 in simple system

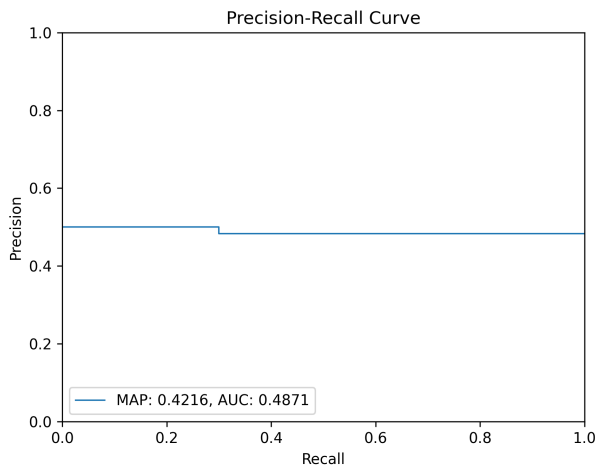


Figure 21: Precision-recall curve for query 5 in advanced system

8.6 Overall System Evaluation

Systems:

Global	Simple System	Advanced System
MAP	0.3848	0.7632

Table 17: MAP averages for both systems

Based on these averages, we conclude that boosting the schema and queries significantly improves performance. This enhancement not only yields better precision in the top 10 results but also leads to noticeably higher overall average metrics across all queries.

Some of the queries have text in English because having text in Portuguese gave us almost no results. For example, in query 2, we needed to replace "número 1" with "number 1" because the option with Portuguese text gave us less than ten results, each of which didn't really allow us to evaluate the query result efficiently. With the replacement of the Portuguese text with an English one, we got over 50 results, and that granted us a better evaluation because we got a lot more documents to evaluate.

9 Conclusion

In the end, we have a solid basis for a retrieval system that allows users to easily find their desired music/music type based on theme or even music characteristics. This sets a clear base on which we can then expand the project to achieve what ultimately led us to start it in the first place, helping other people find new and old favourite songs. Of course, in this work, we can detect some flaws in our system, which we will focus on enhancing in future work. These steps were quite challenging overall, since finding the right dataset, deciding which part of the dataset would be used while keeping a substantial enough size, and uniting the prepared dataset with the API used to obtain extra information were all challenging. However, with a good preparation of the data, we overcame these difficulties. Also, the lack of filters for Portuguese in Solr was quite challenging, as we had to adapt to the existing filters and work with a small set.

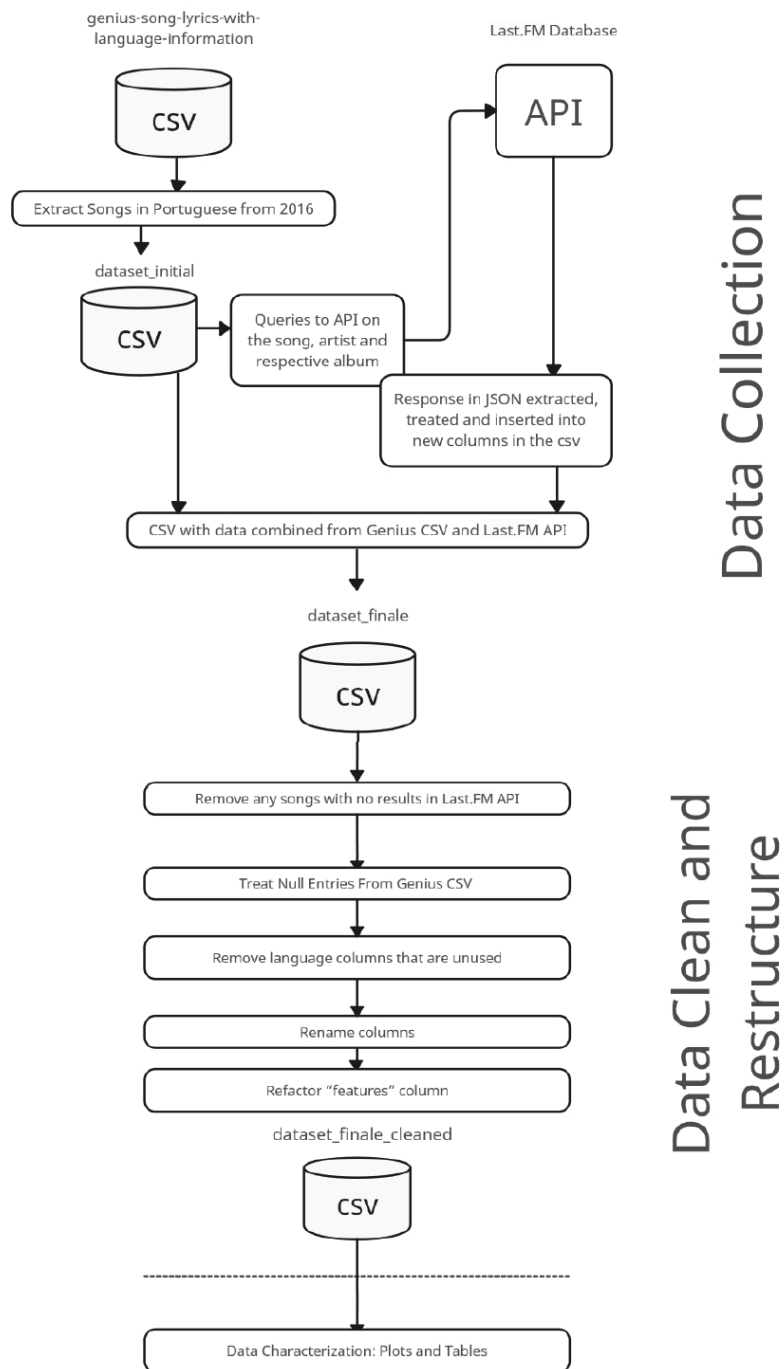
References

- [1] Steven Bird, Edward Loper, and Ewan Klein. 2023. Natural Language Toolkit. <https://www.nltk.org/> Version 3.6.7.
- [2] Carlos Garcia. 2022. Genius Song Lyrics with Language Information. <https://www.kaggle.com/datasets/carlosgdcj/genius-song-lyrics-with-language-information>. Accessed: 2024-10-13.
- [3] Last.fm. 2024. Last.fm API. <https://www.last.fm/api>. Accessed: 2024-10-13.
- [4] Andreas Mueller. 2018. wordcloud: A little word cloud generator in Python. https://github.com/amueller/word_cloud
- [5] Matplotlib Development Team. 2024. Matplotlib Documentation. <https://matplotlib.org/>. Accessed: 2024-10-13.
- [6] NumPy Development Team. 2024. NumPy Documentation. <https://numpy.org/>. Accessed: 2024-10-13.
- [7] Pandas Development Team. 2024. Pandas Documentation. <https://pandas.pydata.org/>. Accessed: 2024-10-13.

[1? ? -7]. [?]

A Annexes

Received 11 October 2024; revised 16 October 2024

**Figure 22: Data Pipeline**